

MILESTONE 3 · SPÉCIFICATIONS

Spécifications fonctionnelles et techniques

Fonctionnalités par profil, choix technologiques justifiés et architecture globale

Projet **PlantCare AI** — Big Data / IA · Groupe **MSI-5-26-BD** · Équipe : **Tina, Kaavya, Nathan, Thomas** · Client : Zineb Lamri · **Juillet 2026**

Ce document décrit ce que le système PlantCare AI permet de faire, pour chaque profil d'utilisateur, puis comment il est construit sur le plan technique, en justifiant chaque choix technologique et en présentant l'architecture globale de la solution.

1. Spécifications fonctionnelles par profil

Le système s'adresse d'abord à un **utilisateur final**, particulier possédant des plantes d'intérieur. Celui-ci peut enregistrer ses plantes en précisant leur espèce, leur emplacement et la taille du pot, puis téléverser une photographie pour que le système identifie automatiquement l'espèce concernée. Il peut consulter, pour chacune de ses plantes, une recommandation d'arrosage indiquant s'il faut arroser maintenant, vérifier prochainement ou s'abstenir, ainsi qu'un conseil d'entretien rédigé en langage naturel qui tient compte de la météo du jour et de l'état mesuré. Il accède enfin à l'historique des soins déjà réalisés.

Le système s'adresse ensuite à un **data steward**, ou administrateur, chargé de la supervision. Ce profil surveille la qualité des données circulant dans le datalake, vérifie la bonne classification des sources, peut relancer le pipeline d'ingestion en cas d'incident et consulte les métriques de performance des modèles.

Un troisième acteur, le **système ordonnanceur**, agit de façon automatique : il déclenche périodiquement l'ingestion des données météorologiques et des mesures de capteurs, puis met à jour les recommandations sans intervention humaine.

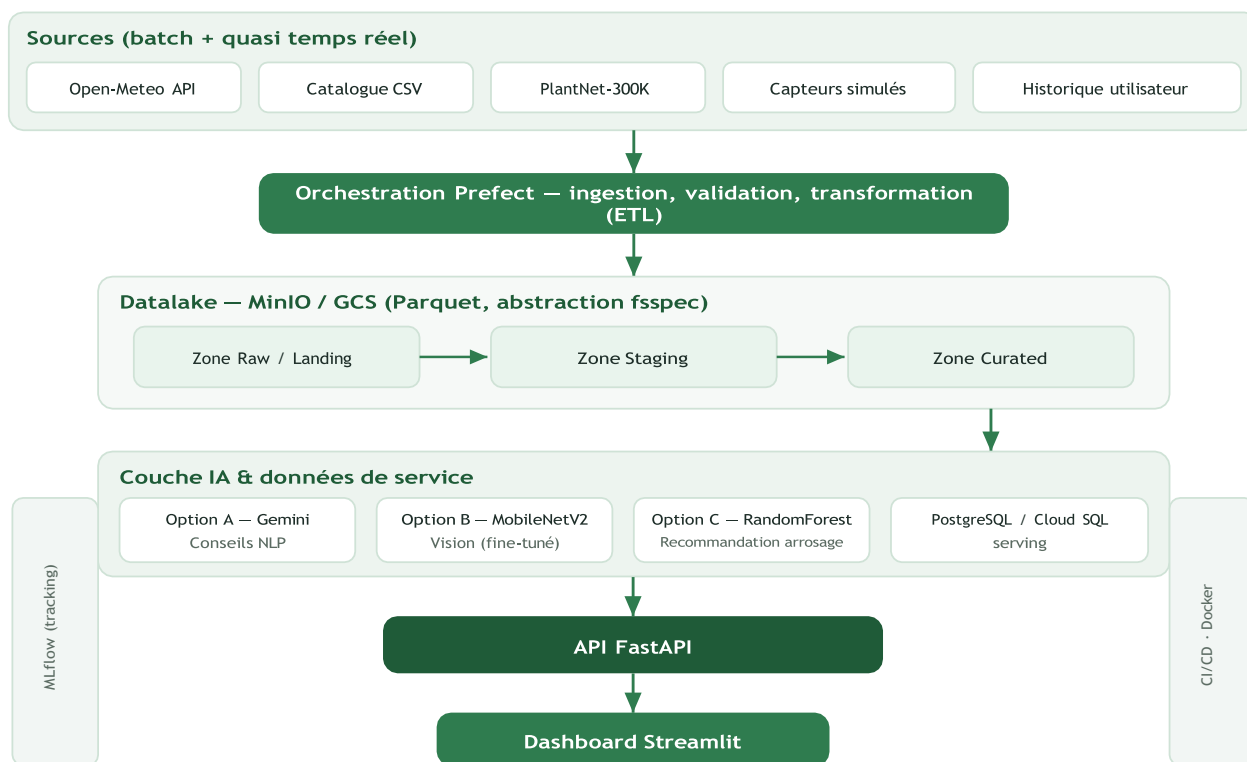
2. Spécifications techniques

L'architecture retenue est une chaîne data complète, pensée pour être **portable entre l'exécution locale et le cloud**. L'orchestration est confiée à **Prefect**, préféré à Airflow parce qu'il s'écrit nativement en Python et ne nécessite pas de composants lourds à conteneuriser, ce qui convient à un POC de vingt-huit heures. Le datalake s'appuie sur un stockage objet compatible S3, à savoir **MinIO** en local et **Cloud Storage** en cloud, l'accès étant abstrait par la bibliothèque fsspec de sorte que le même code manipule des chemins locaux ou distants selon une simple variable d'environnement. Les données de service sont stockées dans **PostgreSQL**, remplaçable par Cloud SQL. Le suivi des expérimentations et le versionnement des modèles reposent sur **MLflow**.

Les trois fonctionnalités d'IA mobilisent des technologies adaptées à leur nature. L'option A appelle un grand modèle de langage via une passerelle abstraite, dont l'implémentation par défaut est **Vertex AI Gemini**, ce qui garde l'IA générative dans l'écosystème Google. L'option B repose sur un réseau de vision **MobileNetV2** pré-entraîné puis affiné sur un sous-ensemble d'espèces issu de PlantNet-300K, choisi pour sa légèreté qui autorise un fine-tuning rapide même sans carte graphique. L'option C entraîne un modèle maison de type **RandomForest**, retenu pour son interprétabilité et sa rapidité d'entraînement. La couche de restitution associe une **API FastAPI** organisée selon les principes de la Clean Architecture et un **dashboard Streamlit**. L'ensemble est conteneurisé avec **Docker Compose** et validé par un pipeline **GitHub Actions**.

Les contraintes de performance visées restent volontairement modestes et réalistes pour un POC : un temps de réponse de l'API inférieur à deux secondes pour une recommandation, une inférence d'image de l'ordre de la seconde sur processeur, une ingestion des sources batch en traitement nocturne et un rafraîchissement des mesures de capteurs par micro-lots de quelques minutes.

3. Schéma d'architecture globale



Architecture globale de PlantCare AI, des sources multi-natures jusqu'au dashboard de restitution.